UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 10: Introduction to ML; Inference and Training in DNNs

Volodymyr Kindratenko

2/19/2026

Course Reminders

- Lab updates
  - Lab 4 is due this week on **Friday**
- Project milestone 1 will be released soon
- Take a note of Midterm 1 date/time
  - **When:** March 3rd, 7-10pm

Today's Objectives

- To learn the basic approach to feedforward neural networks:
  - neural model
  - common functions
  - training through gradient descent

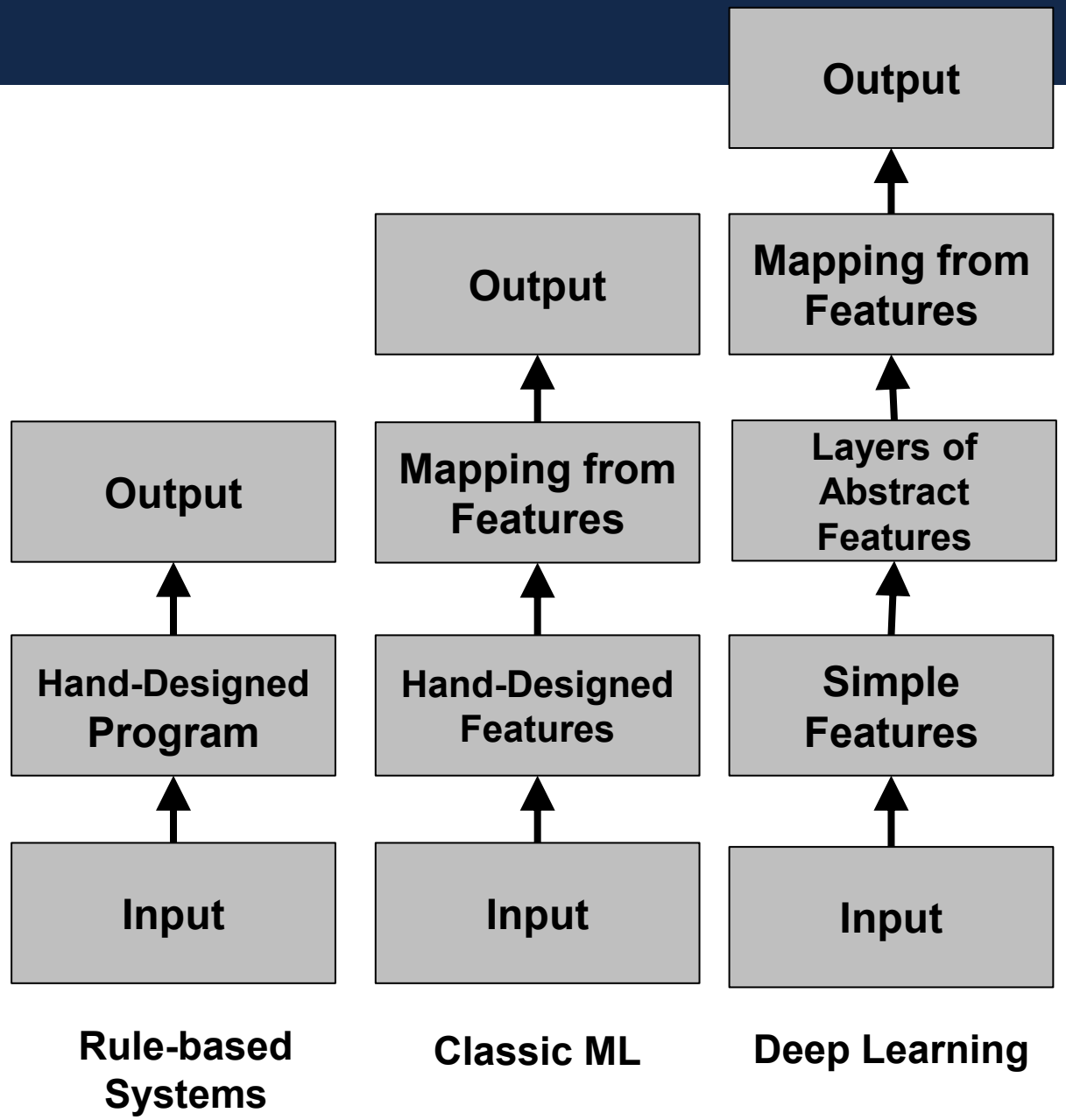
What is Machine Learning?

- **Machine learning**: important method of building applications whose logic is not fully understood
- Typically, **by example**:
  - **use labeled data** (matched input-output pairs)
  - **to represent** desired **relationship**
- **Iteratively adjust program logic** to produce desired/approximate answers (called **training**)
- Types of Learning Tasks
  - Classification, Regression <- structured data
  - Transcription, Translation <- unstructured data
  - …

Why Machine Learning Now?

- **Computing Power**
  - GPU computing hardware and programming interfaces such as CUDA has enabled very fast research cycle of deep neural net training
- **Data**
  - Lots of cheap sensors, cloud storage, IoT, photo sharing, etc.
- **Needs**
  - Autonomous Vehicles, Smart Devices, Security, Societal Comfort with Tech, Health Care

Evolution of ML



Let's Look at Classification

- In a **classification problem**, we model
  - a function mapping an input vector to a set of ***C*** categories: $F : \mathbb{R}^N \rightarrow \{1, \ldots, C\}$,
  - where the function ***F* is unknown**.
- We **approximate *F* using a set of functions *f***
  - parametrized by a (large) set of weights, Θ
  - that map from a vector of ***N*** real values* to an integer value representing a category: for category *i*, **prob*(i) = f(x, Θ)***

*floating-point values

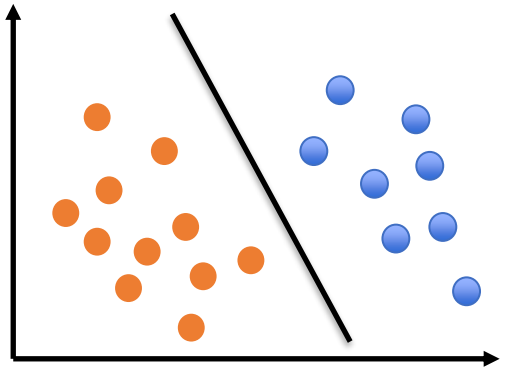
Linear Classifier (Perceptron)

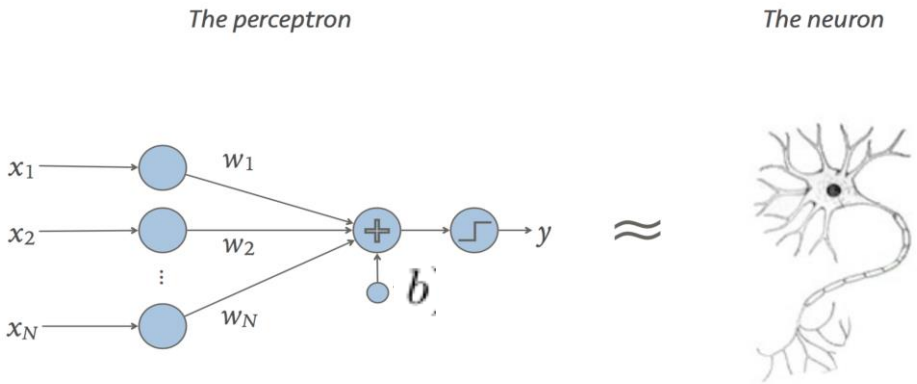
- Our formulation:

$$y = f(\mathbf{x}, \Theta)$$

$$\Theta = \{W, b\}$$

$$y = sign(W \bullet \mathbf{x} + b)$$





- Dot product + Scalar addition:

$$y = W \bullet \mathbf{x} + b$$

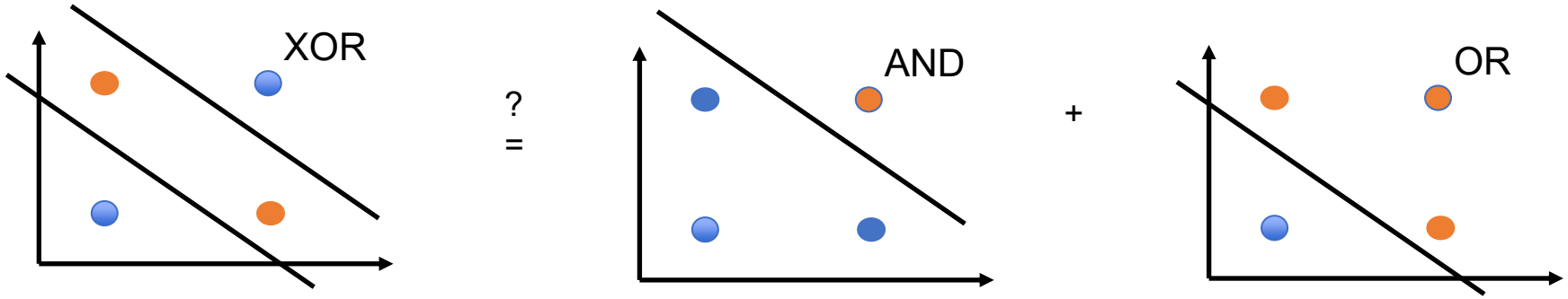
output

bias

weight

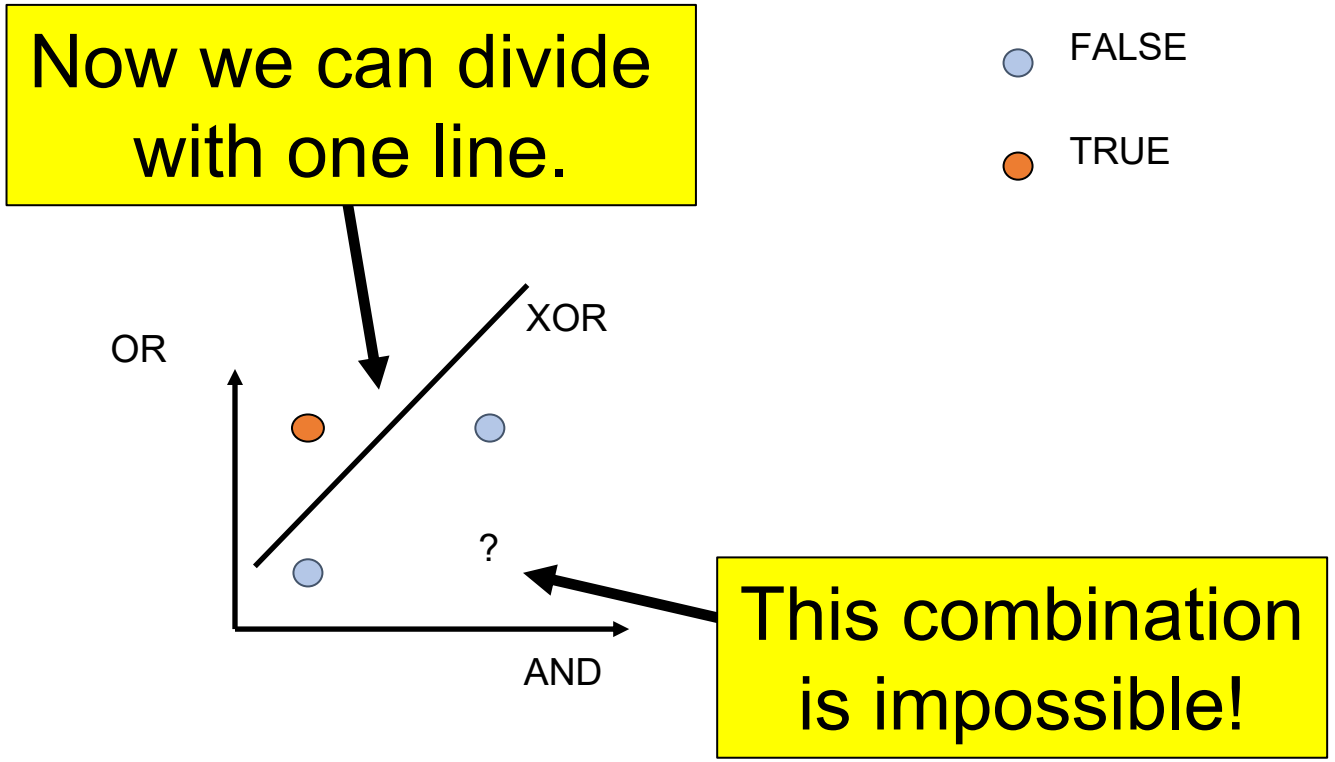
input

Can We Learn XOR with a Perceptron?

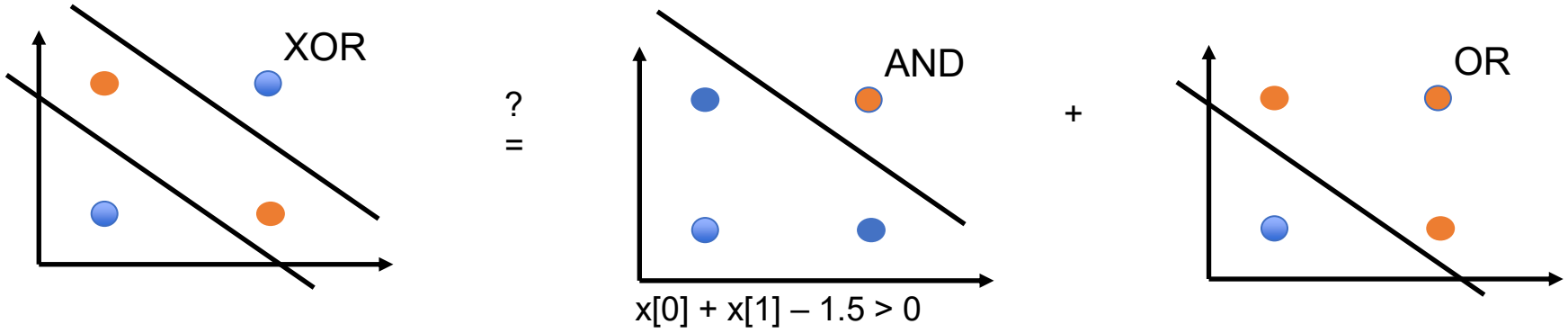


Multiple Layers Solve More Problems

- **What if input dimensions are AND and OR?**



Perceptron



x[1]	x[0]	AND	OR	XOR
0	0	-1 (-1.5 < 0)	-1 (-0.5 < 0)	-1 (-2.0 < 0)
0	1	-1 (-0.5 < 0)	1 (0.5 > 0)	?
1	0	-1 (-0.5 < 0)	1 (0.5 > 0)	?
1	1	1 (0.5 > 0)	1 (1.5 > 0)	1 (2.0 > 0)

XOR is not a linear combination of AND and OR functions.

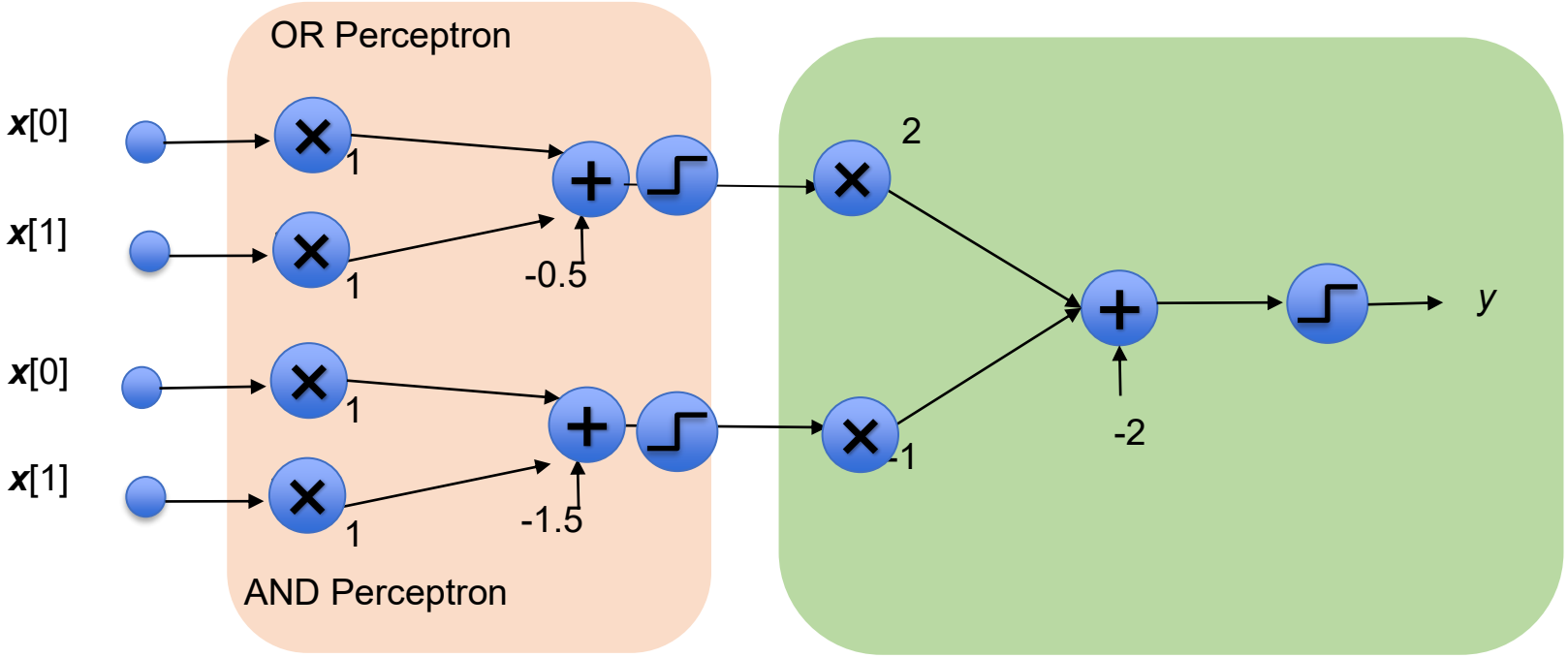
Multi-Layer Perceptron

x[1]	x[0]	AND	OR	XOR
0	0	-1	-1	-1 (-3 < 0)
0	1	-1	+1	1 (1 > 0)
1	0	-1	+1	1 (1 > 0)
1	1	+1	+1	-1 (-1 < 0)

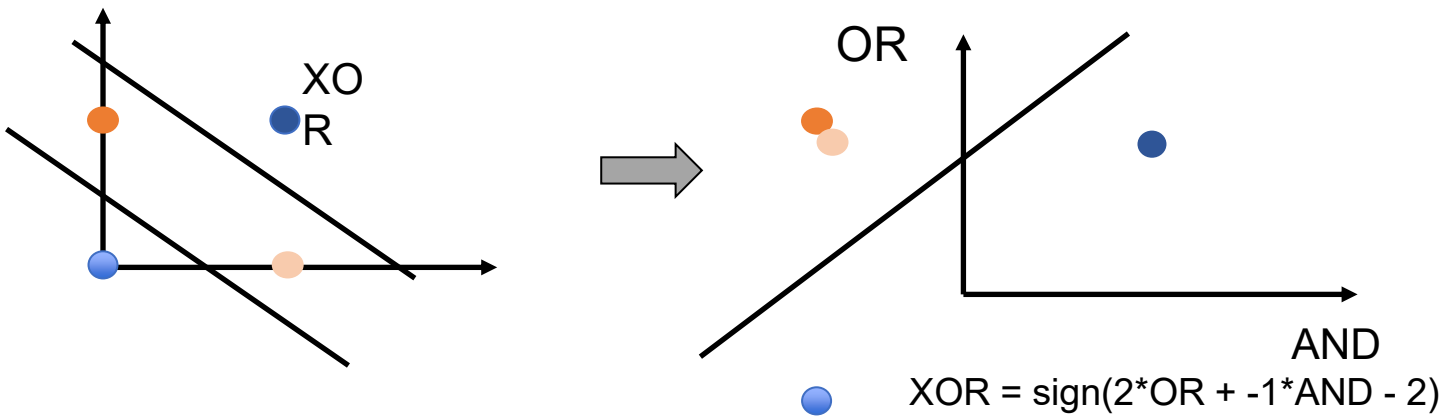
OR = sign(***x***[0] + ***x***[1] - 0.5)

AND = sign(***x***[0] + ***x***[1] - 1.5)

sign() function adds non-linearity to "reposition" data points for the next layer.

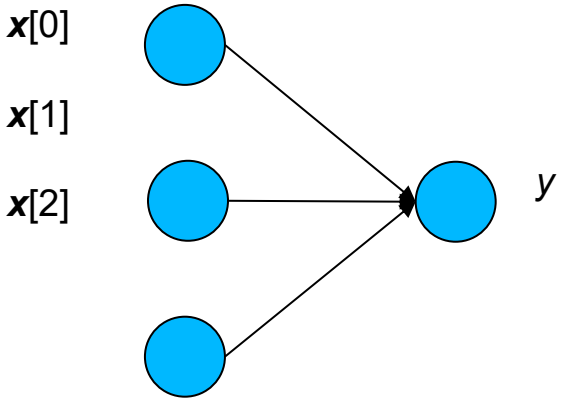


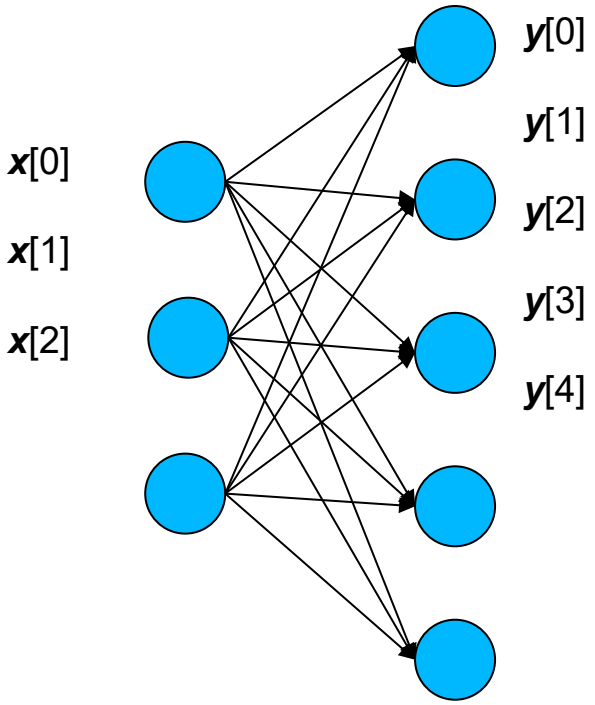
Multi-Layer Perceptron – data repositioning



x[1]	x[0]	AND	OR	XOR
0	0	-1	-1	-1 (-3 < 0)
0	1	-1	+1	1 (1 > 0)
1	0	-1	+1	1 (1 > 0)
1	1	+1	+1	-1 (-1 < 0)

Generalize to Fully-Connected Layer

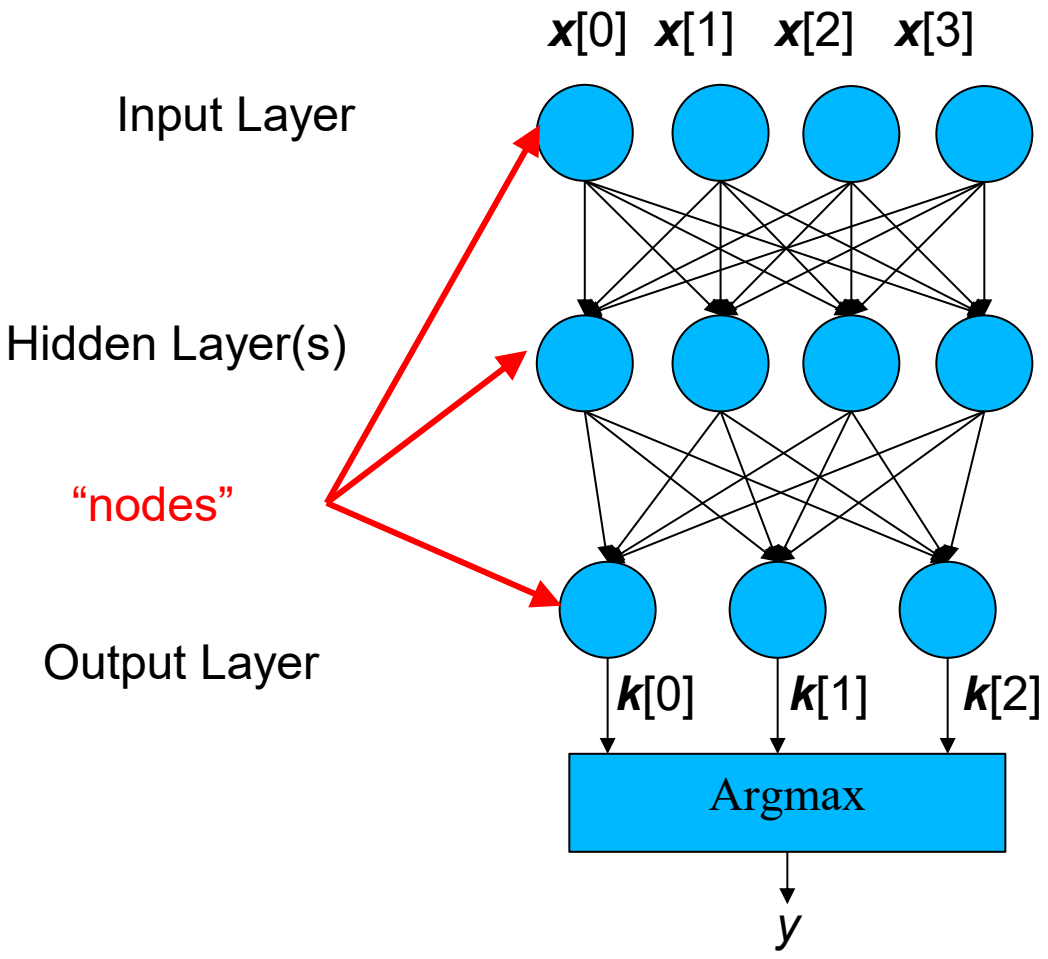




Linear Classifier:
Input vector ***x***×weight vector ***w*** to produce scalar output *y*

Fully-connected:
Input vector ***x***×weight <u>matrix</u> ***w*** to produce <u>vector</u> output *y*

Multilayer Terminology



W_k[i, j] : weight between i^{th} input and j^{th} output of the kth layer

W_1 is [4x4], b_1 is [4x1]

W_2 is [4x3], b_2 is [3x1]

Probability that input is class ***k***[i]

How to Determine the weights?

- Look at observational data to determine the weights?
- Pick some random values?
- Start with something that partially works?

- With enough *labeled* data, we can automatically *encode* the relationship between inputs and outputs.

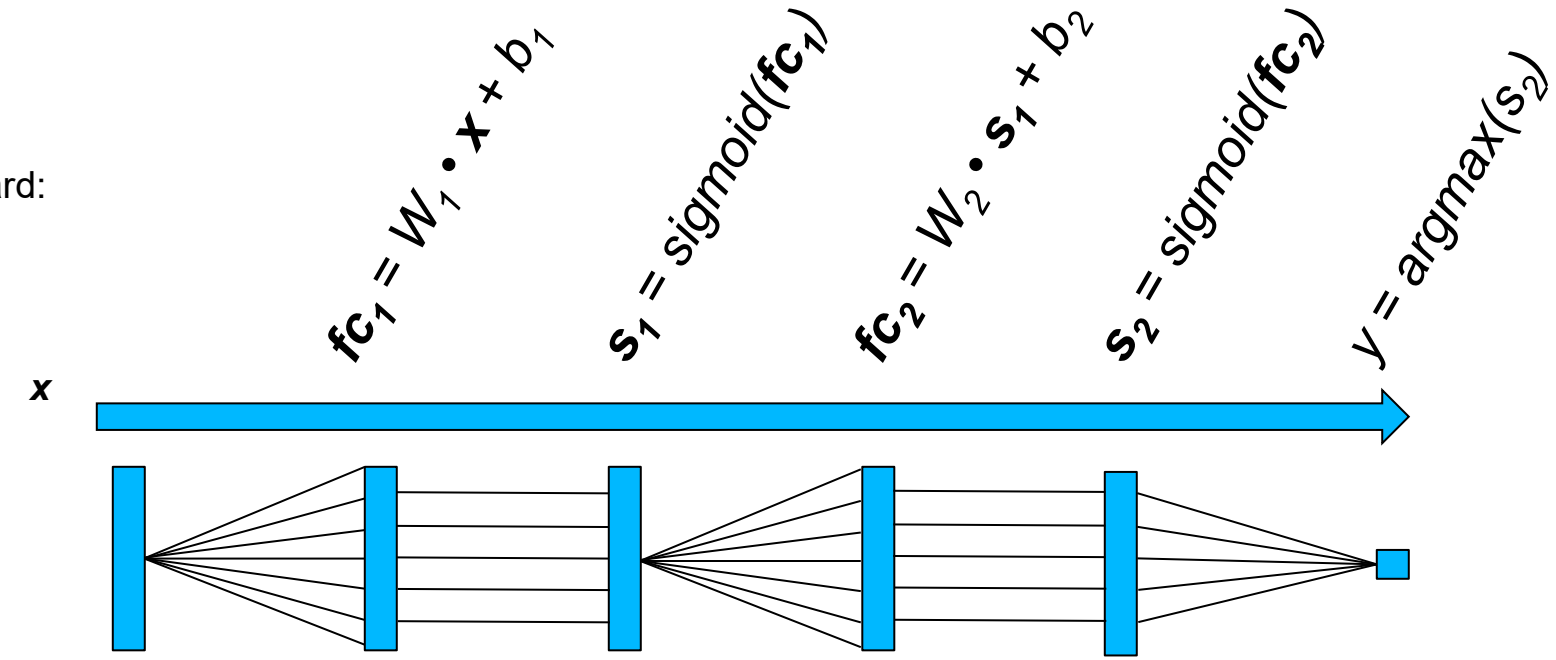
Forward and Backward Propagation

- Forward (inference)
  - Given parameters Θ and input $\boldsymbol{x}$, produce label y

- Backward (training)
  - Need a way to assess correctness (loss function)
  - Example: $(x - y)^2$
  - Find Θ, such that loss is minimized over all input data

Forward Propagation (Inference)

Forward:

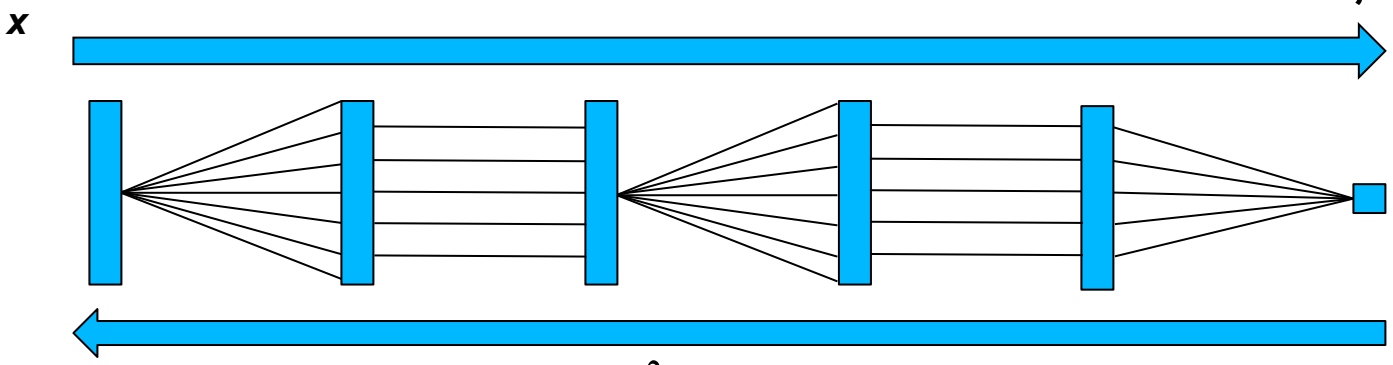


Backward Propagation (Training)

Forward:

$\mathbf{fc_1} = W_1 \bullet \mathbf{x} + b_1$

$y = \text{argmax}(s_2)$



Minimize loss over entire training set:

Need $\frac{dE}{dW}$

Backward:

$$\frac{dE}{dfc_1} = \frac{dE}{ds_1}\frac{ds_1}{dfc_1}$$

$$\frac{dE}{ds_1} = \frac{dE}{dfc_2}\frac{dfc_2}{ds_1}$$

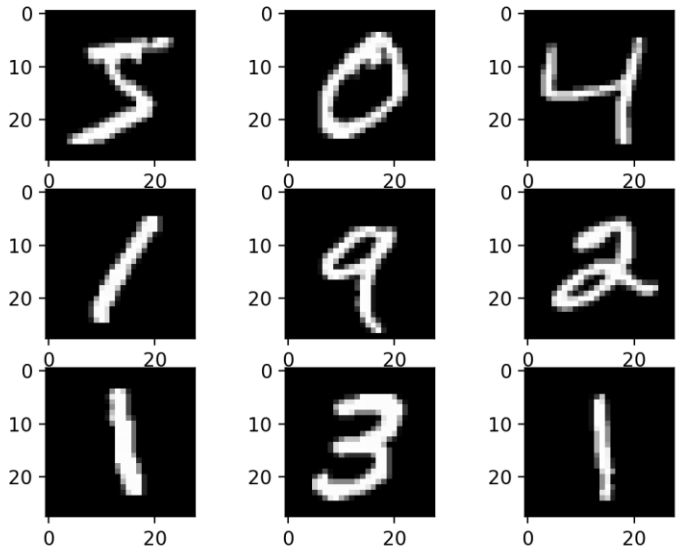
$$\frac{dE}{dfc_2} = \frac{dE}{ds_2}\frac{ds_2}{dfc_2}$$

$$\frac{dE}{ds_2} = \frac{dE}{dy}\frac{dy}{ds_2}$$

$$\frac{dE}{dy}$$

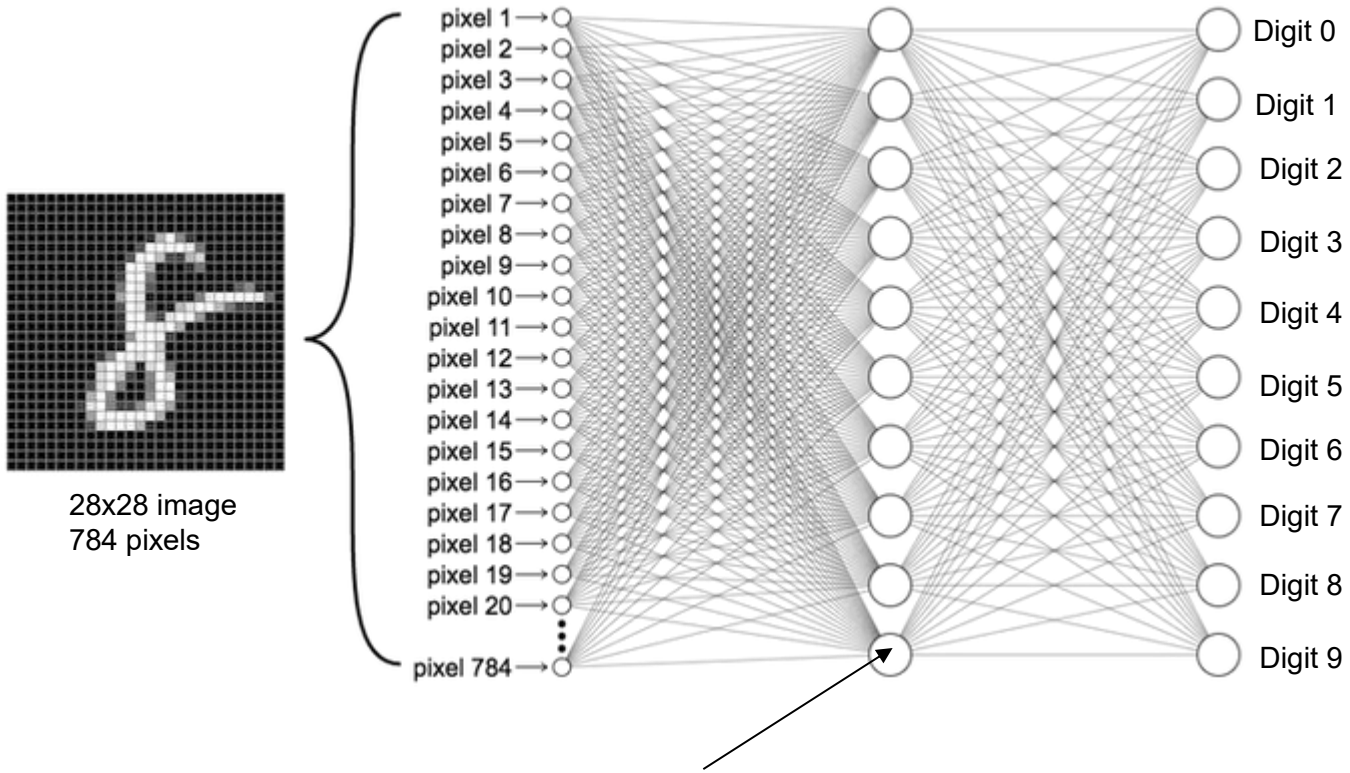
Example: Digit Recognition

- Let's consider an example.
  - **handwritten digit recognition**:
  - given a **28×28 grayscale image**,
  - produce a **number from 0 to 9**.
- Input dataset
  - **60,000** images
  - Each labeled by a human with correct answer.



MNIST dataset

Multi-Layer Perceptron (MLP) for Digit Recognition



n_k^1 = activation($\boldsymbol{w}_k \boldsymbol{x}$+$b_k$)

This network has
- 784 nodes on input layer (L0)
- 10 nodes on hidden layer (L1)
- 10 nodes on output layer (L2)

784*10 weights + 10 biases for L1
10*10 weights + 10 biases for L2

A total of 7,960 parameters

Each node represents a function, based on a linear combination of inputs + bias

Activation function “repositions” output value.

Sigmoid, sign, ReLU are common…

How Do We Determine the Weights?

- **First layer** of perceptron:
  - **784** (28^2) inputs, **10** outputs, **fully connected**
  - **[10×784]** weight matrix ***W***
  - **[10 x 1]** bias vector ***b***
- **Use labeled training data to pick weights.**
- Idea:
  - given enough labeled input data,
  - we can **approximate the input-output function**.

Forward and Backward Propagation

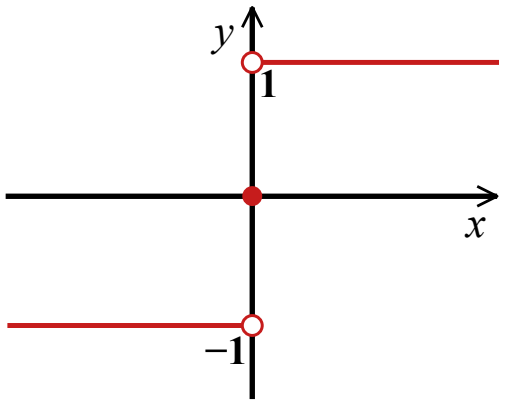
- Forward (**inference**):
  - given input ***x*** (for example, an image),
  - **use parameters *Θ*** (***W*** and *b* for each layer)
  - **to compute probabilities *k[i]*** (ex: for each digit *i*).

- Backward (**training**):
  - given input ***x***, parameters ***Θ***, and outputs ***k[i]***,
  - **compute error *E*** based on target label ***t***,
  - then **adjust *Θ*** proportional to ***E*** to reduce error.

Neural Functions Impact Training

- Recall perceptron function: **y = sign (W·x + b)**

- **To propagate error** backwards,
  - **use chain rule** from calculus.
  - **Smooth functions are useful.**



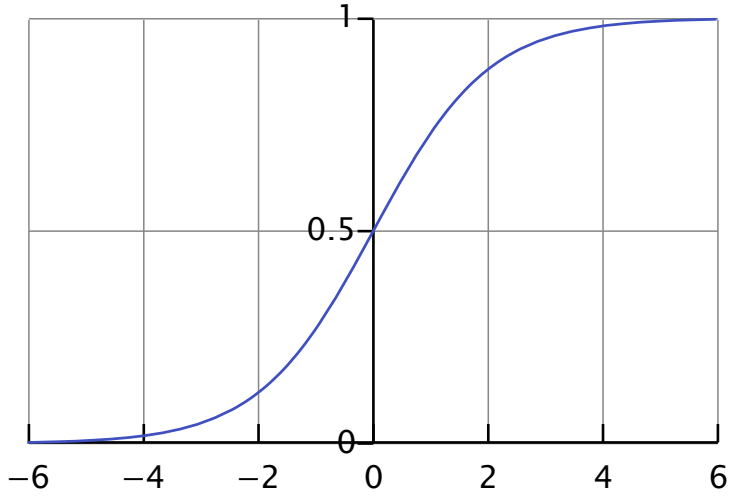
Sign is not a smooth function.

One Choice: Sigmoid/Logistic Function

- Until about 2017,
  - **sigmoid / logistic function** most popular

$$\boldsymbol{f(x) = \frac{1}{1+e^{-x}}} \qquad (f: \mathbb{R} \rightarrow (0,1))$$

for replacing sign.



- Once we have *f(x)*, finding *df/dx* is easy:

$$\boldsymbol{\frac{df(x)}{dx}} = \frac{e^{-x}}{(1+e^{-x})^2} = f(x)\frac{e^{-x}}{(1+e^{-x})} = \boldsymbol{f(x)(1 - f(x))}$$

(Our example used this function.)

Today's Choice: ReLU

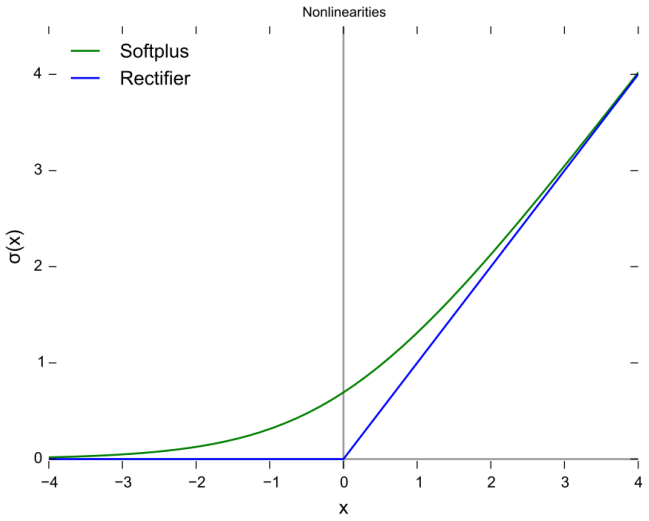
- In 2017, most common choice became
  - **rectified linear unit / ReLU / ramp function**

    $$\boldsymbol{f(x) = \max(0, x)} \quad (f: \mathbb{R} \rightarrow \mathbb{R}^+)$$

    which is much faster (no exponent required).
- A smooth approximation is **softplus/SmoothReLU**

  $$\boldsymbol{f(x) = \ln(1 + e^x)} \quad (f: \mathbb{R} \rightarrow \mathbb{R}^+)$$

  which is the integral of the logistic function.



- Lots of variations exist. See for example Wikipedia for an overview and discussion of tradeoffs.

Use Softmax to Produce Probabilities

How can sigmoid / ReLU produce probabilities?

- They can't.
  - Instead, given output vector **Z = (z[0], …, z[C-1])***,
  - we produce a second vector **K = (k[0], …, k[C-1])**
  - using the **softmax function**

$$k[i] = \frac{e^{z[i]}}{\sum_{j=0}^{C-1} e^{z[j]}}$$

Notice that **the k[i] sum to 1.**

*Remember that we classify into one of C categories.

Softmax Derivatives Needed to Train

- We also need the **derivatives of softmax**,

$$\frac{dk[i]}{dz[m]} = k[i](\delta_{i,m} - k[m]),$$

where $\delta_{i,m}$ is the Kronecker delta
(1 if i = m, and 0 otherwise).

Forward and Backward Propagation

Forward:

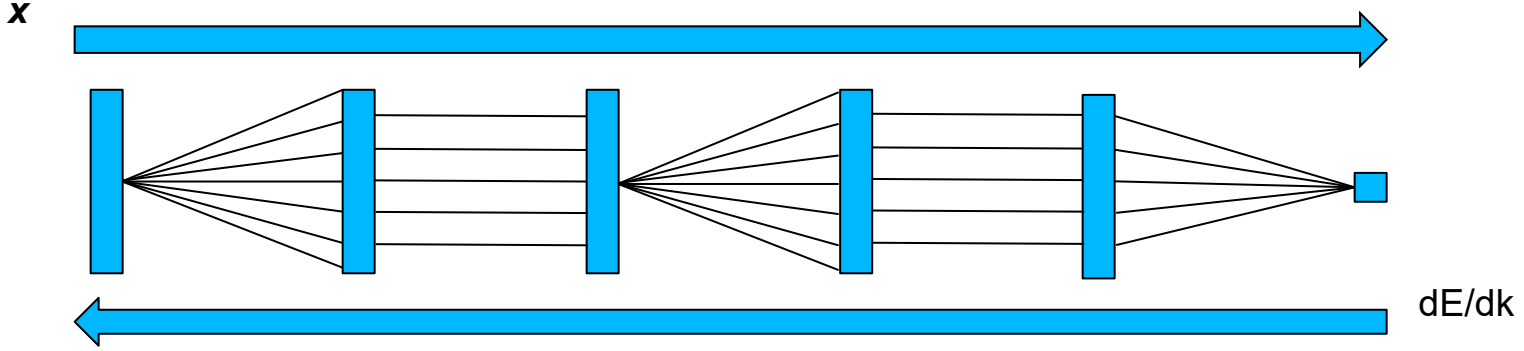
$\mathbf{fc_1} = W_1 \bullet \mathbf{x} + b_1$

$\mathbf{y} = \text{sigmoid}(\mathbf{fc_1})$

$\mathbf{fc_2} = W_2 \bullet y + b_2$

$\mathbf{z} = \text{sigmoid}(\mathbf{fc_2})$

$\mathbf{k} = \text{softmax}(z)$



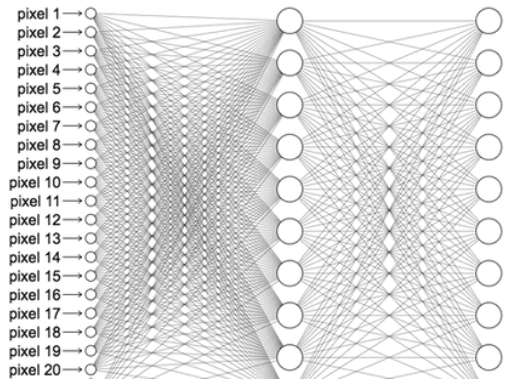
Backward:

$$\frac{dE}{dfc_1} = \frac{dE}{dy}\frac{dy}{dfc_1}$$

$$\frac{dE}{dy} = \frac{dE}{dfc_2}\frac{dfc_2}{dy}$$

$$\frac{dE}{dfc_2} = \frac{dE}{dz}\frac{dz}{dfc_2}$$

$$\frac{dE}{dz} = \frac{dE}{dk}\frac{dk}{dz}$$



Choosing an Error Function

- Many error functions are possible.
- For example, **given label *T*** (digit ***T***),
  - ***E = 1 – k[T]***, the **probability of not classifying as *t***.
- **Alternatively**, since our categories are numeric, we can **penalize quadratically**:

$$E = \sum_{j=0}^{C-1} k[j](j - T)^2$$

- Let's **go with the latter**.

Stochastic Gradient Descent

How do we calculate the weights?

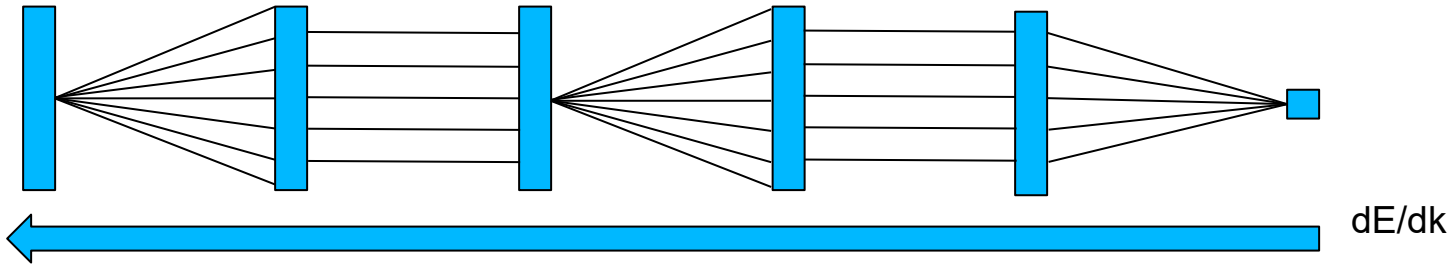
- One common answer: **stochastic gradient descent**.
  1. **Calculate**
     - **derivative** of sum **of error *E***
     - **over all** training **inputs**
     - **for** all network parameters ***Θ***.
  2. **Change *Θ* slightly** in the opposite direction (to decrease error).
  3. **Repeat**.

Stochastic Gradient Descent

- More precisely,
  1. **For every input *X***,
  2. evaluate network to **compute *k[i]*** (forward),
  3. then **use *k[i]* and label *T*** (target digit) **to compute error *E***.
  4. Backpropagate error derivative to **find derivatives for each parameter**.
  5. **Adjust Θ to reduce total *E*: $\Theta_{i+1} = \Theta_i - \varepsilon\Delta\Theta$**

Parameter Updates and Propagation

$$fc_1 = W_1 \bullet x + b_1$$



$$\frac{dE}{dfc_1} = \frac{dE}{dy}\frac{dy}{dfc_1} \qquad \frac{dE}{dy} = \frac{dE}{dfc_2}\frac{dfc_2}{dy} \qquad \frac{dE}{dfc_2} = \frac{dE}{dz}\frac{dz}{dfc_2} \qquad \frac{dE}{dz} = \frac{dE}{dk}\frac{dk}{dz}$$

Need propagated error gradient (from backward pass)

Weight update

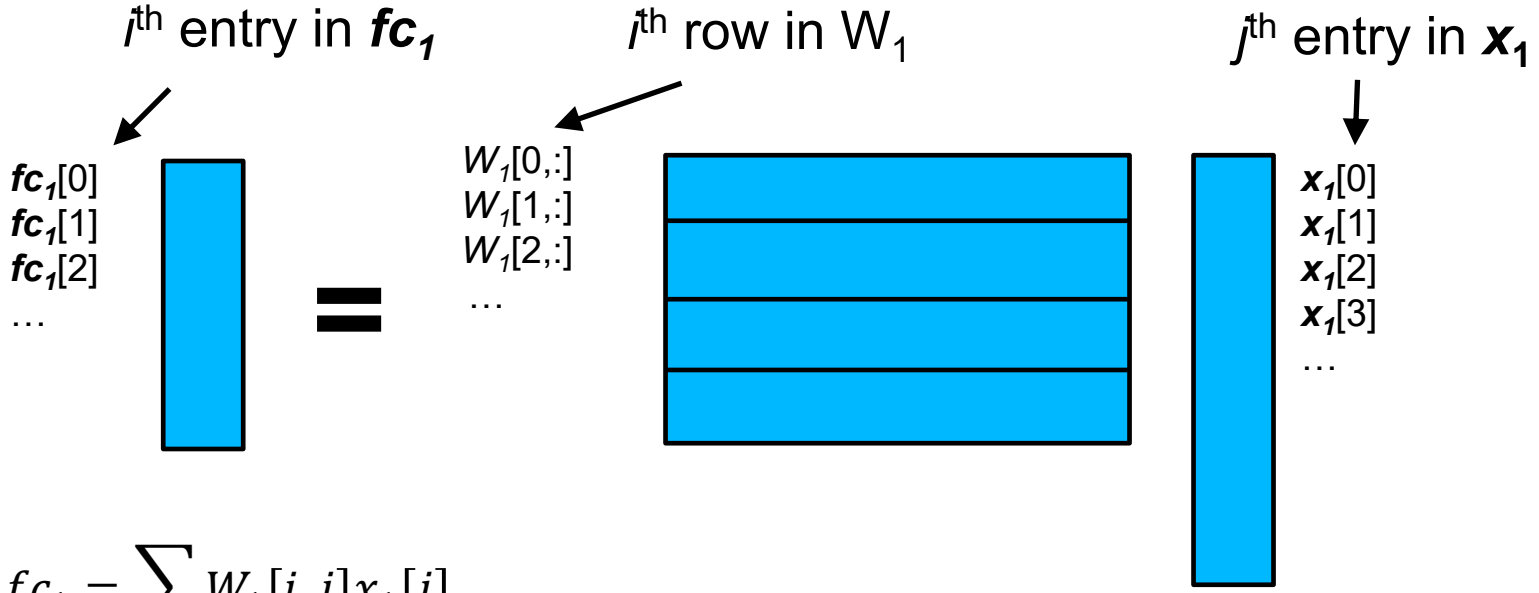
$$\frac{dE}{dW_1} = \frac{dE}{dfc_1}\frac{dfc_1}{dW_1} = \frac{dE}{dfc_1}x$$

Need input (from forward pass)

Example: Gradient Update with One Layer

$\Theta_{i+1} = \Theta_i - \varepsilon\Delta\Theta$	$W_{i+1} = W_i - \varepsilon\Delta W$	Parameter Update
	$y = W \bullet \boldsymbol{x} + b$	Network function
	$\frac{dy}{dW} = x$	Network weight gradient
	$E = ½ (y - t)^2$	Error function
	$\frac{dE}{dy} = y - t = Wx + b - t$	Error function gradient
	$\Delta W = \frac{dE}{dW} = \frac{dE}{dy}\frac{dy}{dW}$	Full weight update expression
	$W_{i+1} = W_i - \varepsilon(W\boldsymbol{x}+b-t)x$	Full weight update term

Fully-Connected Gradient Detail



$$fc_1 = \sum_j W_1[i,j]x_1[j]$$

Computed from previous layer

$$\frac{dE}{dW_1[i,j]} = \frac{dE}{dfc_1[i]}\frac{dfc_1[i]}{dW_1[i,j]} = \frac{dE}{dfc_1[i]}x_1[j]$$

Need input to this layer

Batched Stochastic Gradient Descent

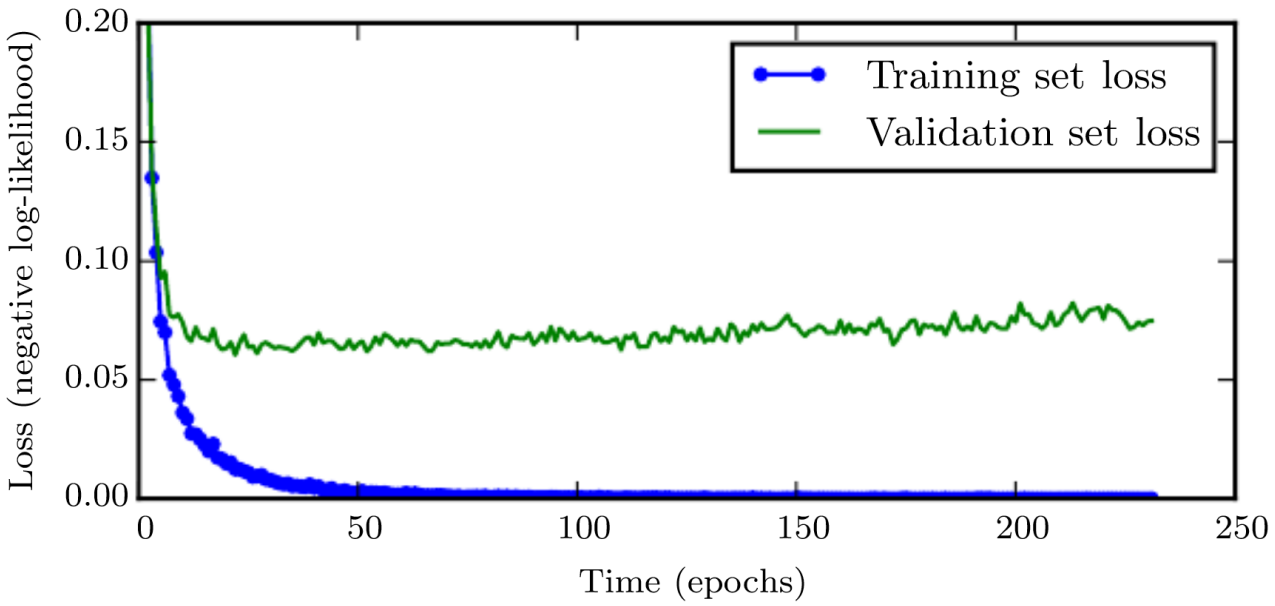
- A training *epoch* (a pass through whole training set)
  - *Set* $\Delta\Theta = 0$
  - For each labeled image:
    - Read data to initialize input layer
    - Evaluate network to get y (forward)
    - Compare with target label t to get error E
    - Backpropagate error derivative to get parameter updates
    - Accumulate parameter updates into $\Delta\Theta$
  - $\Theta_{i+1} = \Theta_i - \varepsilon\Delta\Theta$
- Aggregate gradient update most accurately reflects true gradient

Mini-batch Stochastic Gradient

- For each batch in training set
  - For each labeled image in batch:
    - Read data to initialize input layer
    - Evaluate network to get *y* (forward)
    - Compare with target label *t* to get error *E*
    - Backpropagate error derivative to get parameter updates
    - Accumulate parameter updates into $\Delta\Theta$
  - $\Theta_{i+1} = \Theta_i - \varepsilon\Delta\Theta$
- Balance between accuracy of gradient estimation and parallelism

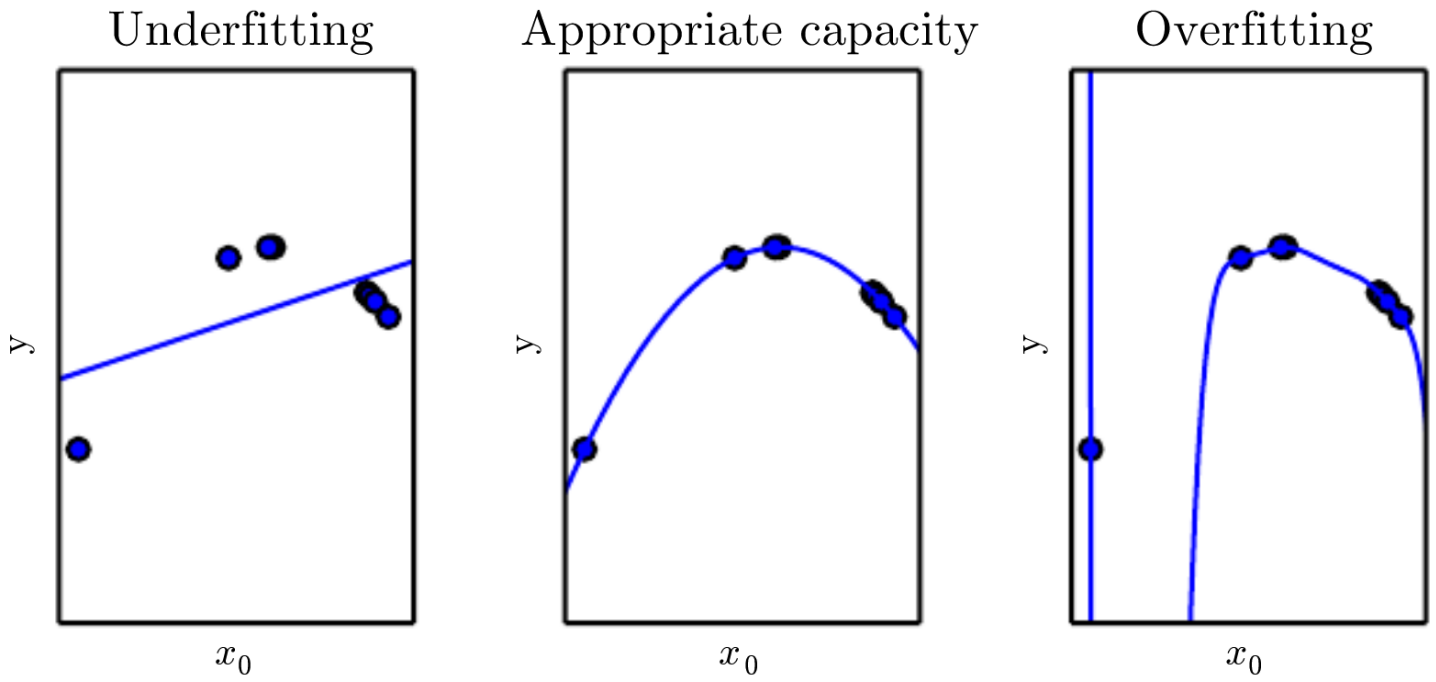
When is Training Done?

- Split labeled data into *training* and *test* sets.
  - Training data to compute parameter updates.
  - Test data to check how model generalizes to new inputs (the ultimate goal!)
- The network can become *too good* at classifying training inputs!



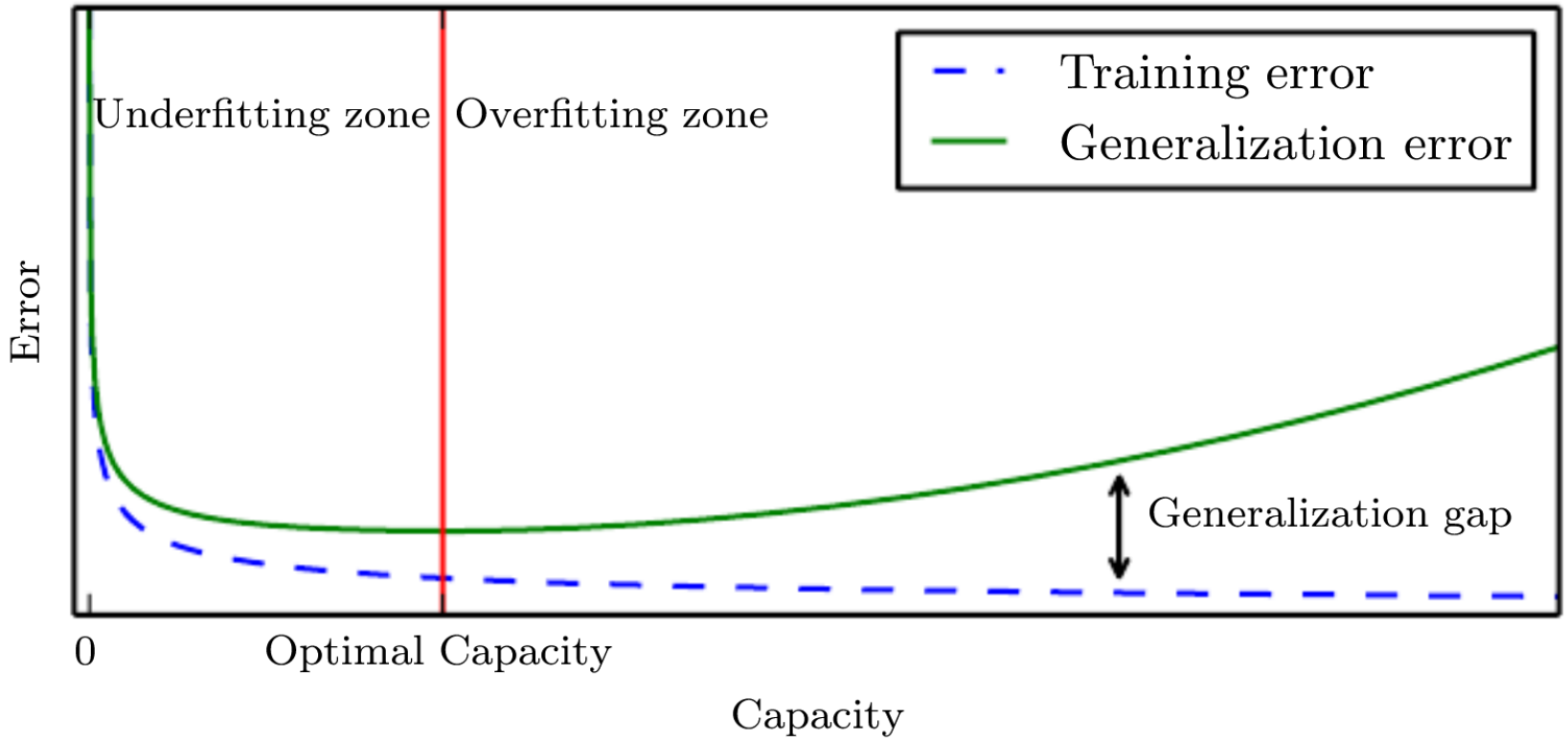
How Complicated Should a Network Be?

- Intuition: like a polynomial fit.
  - High-order terms improve fit, but add unpredictable swings for inputs outside the training set.

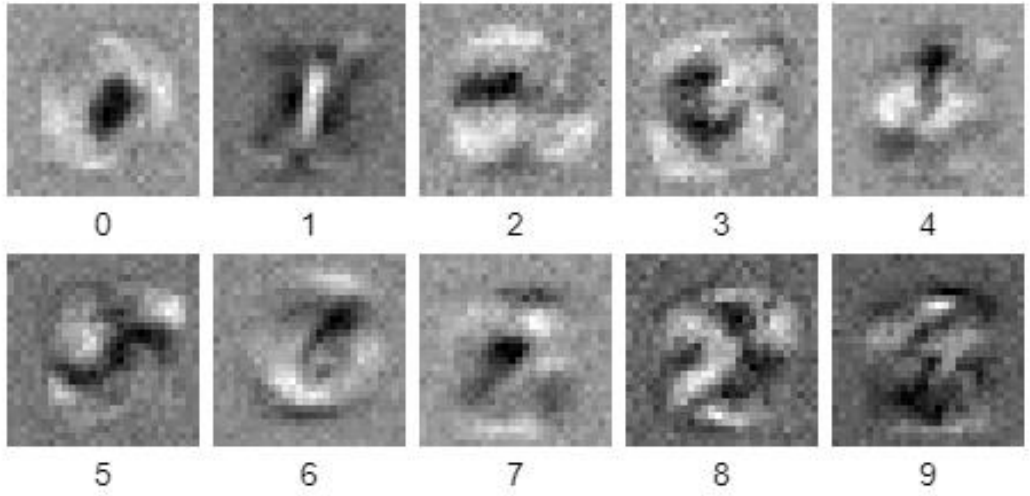


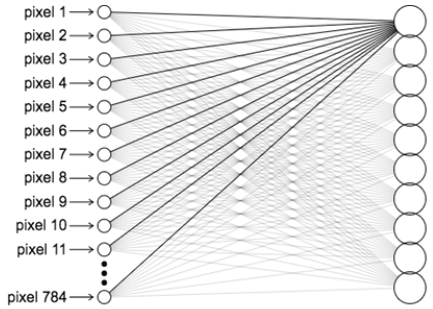
Overtraining Decreases Accuracy

- If network works too well for training data, new inputs cause big unpredictable output changes.

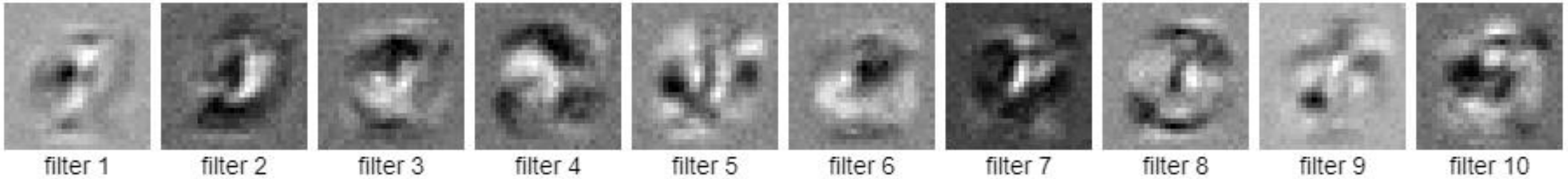


Visualizing Neural Network Weights





MNIST 1st layer



MNIST 2nd layer

From https://ml4a.github.io/ml4a/looking_inside_neural_nets/

No Free Lunch Theorem

- Every classification algorithm has the same error rate when classifying previously unobserved inputs when averaged over all possible input-generating distributions.

- Neural networks must be tuned for specific tasks!

Any More Questions?

Things to Read:
Textbook Chapter 16

Things to Do:
Submit Lab 4